

VELEUČILIŠTE U BJELOVARU
STRUČNI PRIJEDIPLOMSKI STUDIJ MEHATRONIKA
STRUČNI PRIJEDIPLOMSKI STUDIJ RAČUNARSTVO

Tehnička dokumentacija iz predmeta Mikroračunala

Parking senzori

Matko Manzin

Bjelovar, rujan 2025.

Sadržaj

1.	Uvod	1
2.	Opis projektnog zadatka	2
3.	Opis hardvera.....	6
3.1	<i>Tehničke specifikacije mikroupravljača ATmega32u4</i>	<i>6</i>
3.2	<i>USBasp</i>	<i>7</i>
3.3	<i>Ultrazvučni senzor HC-SR04</i>	<i>7</i>
3.4	<i>LCD ekran (GDM1602E).....</i>	<i>9</i>
3.5	<i>Zujalica (eng. buzzer).....</i>	<i>10</i>
3.6	<i>NPN tranzistor S8050.....</i>	<i>10</i>
3.7	<i>LED dioda</i>	<i>11</i>
3.8	<i>Troškovnik</i>	<i>11</i>
4.	Električna shema uređaja	12
5.	Programski kod.....	13
6.	Zaključak.....	20
7.	Literatura	21

1. Uvod

Odabir rada motiviran je stvarnim problemom, tvornički parking senzori na mom vozilu su se pokvarili, pa je cilj bio vlastitom izradom koristeći ATmega32u4 mikroračunalo ostvariti funkcionalnu zamjenu.

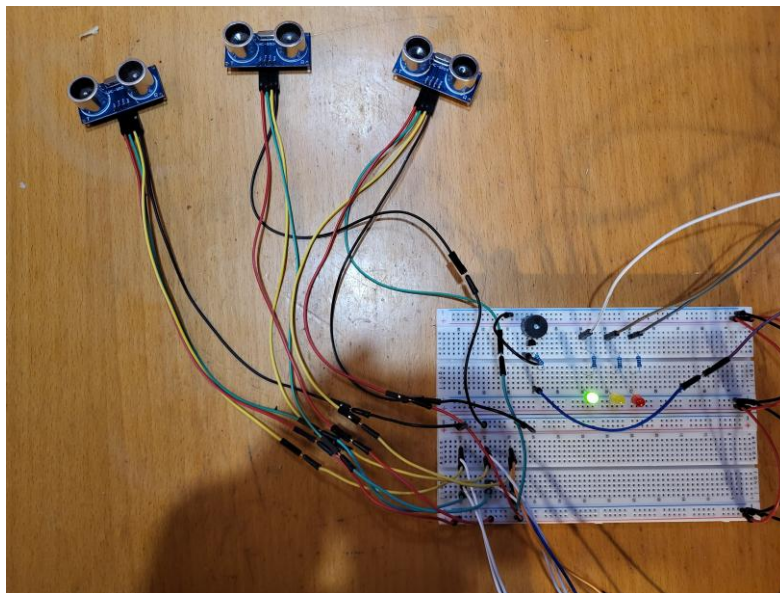
Sustav je automatiziran na način da se tri ultrazvučna senzora HC-SR04 pobuđuju sekvencijalno kratkim impulsom koji traje 10 μ s, svaki u svom vremenskom intervalu. Time se elimira mogućnost da jedan senzor primi impuls drugog senzora. Na temelju izmjerene udaljenosti, uzima se najmanja udaljenost od prepreke i upali jedna od tri LED dioda. Da bi se znalo koja LED dioda će se upaliti definirane su "zone", zona 1 je "zelena zona" i ona ima doseg 60 - 200 cm, zona 2 je "žuta zona" i ona ima doseg 40 - 60 cm, te zona 3 je "crvena zona" i ima doseg 0 - 20 cm. Isto tako radi i zujalica, koja se aktivira na nisko stanje, i na temelju koja je zona aktivna, definiran je interval koliko dugo zujalica mora biti upaljena i ugašena. Na LCD ekranu prikazane su udaljenosti za lijevi (L) senzor, centralni (C) senzor i desni (D) senzor.

Automatizacija u ovom projektu donosi brzu i točnu reakciju sustava uz vrlo nizak trošak i jednostavno održavanje. Sustav samostalno mjeri udaljenost u tri smjera, obrađuje rezultate u stvarnom vremenu i prikazuje ih pomoću boje LED dioda i ritma zujalice, čime smanjuje ljudsku pogrešku i povećava sigurnost pri parkiranju. Iako ultrazvučni senzor HC-SR04 ima ograničenja (ovisnost o materijalu, kutu prepreke i uvjetima okoline), on se može dodatno poboljšati korekcijom temperature.

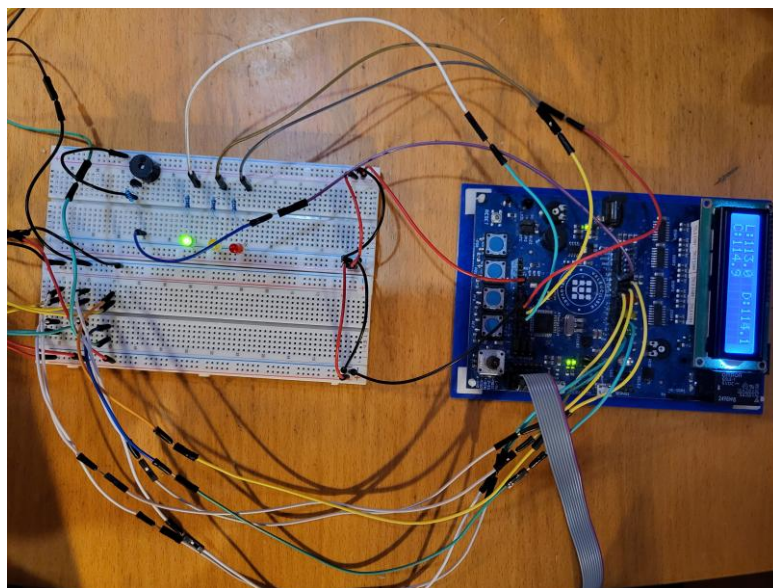
Postojeća rješenja za pomoć pri parkiranju obično koriste ultrazvučne senzore ili kamere, a napredniji sustavi kombiniraju više tehnologija. Ovaj sustav se može poboljšati uvođenjem korekcije temperature što će dovesti do točnijih mjerenja.

2. Opis projektnog zadatka

Sustav parking senzora sastoji se od tri HC-SR04 ultrazvučna senzora koji su podjeljeni na lijevi, centralni i desni senzor. Informacije o udaljenosti se prikazuju na LCD ekranu, ali isto tako se i prikazuju preko LED dioda i pomoću zujalice. Uređaj sekvencijalno pobuđuje svaki senzor 10 μ s TRIG impulsom i mjeri širinu ECHO impulsa u prekidnoj rutini pomoću 16-bitnog timera, timera 1. Iz izmjerenog vremena računa se udaljenost u centimetrima, a od tri vrijednosti uzima se najmanja udaljenost kao udaljenost najbliže prepreke. Postoje tri zone udaljenosti, zelena 60 - 200 cm, žuta 20 - 60 cm, crvena 0 - 20 cm te ovisno o zoni mijenja se LED dioda i ritam zujalice.



Slika 2.1. Parking senzori



Slika 2.2. Parking senzori

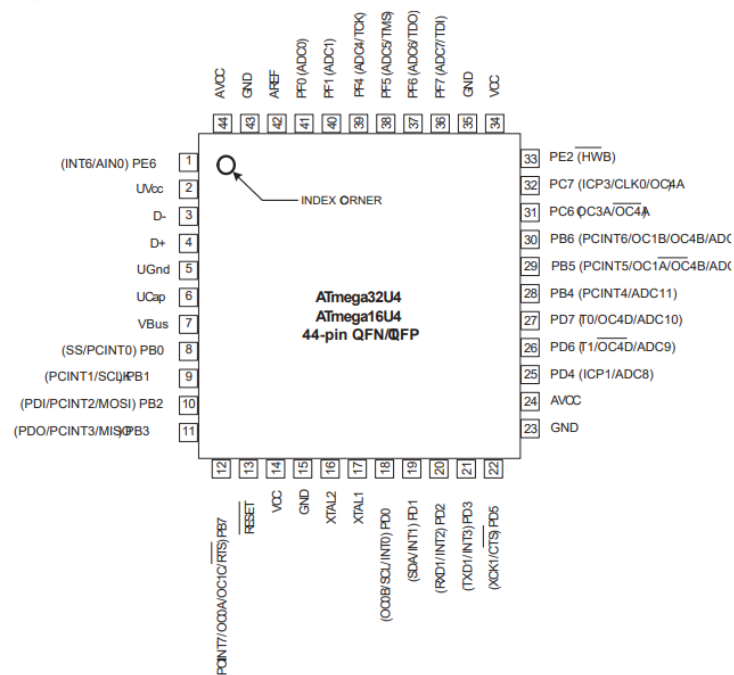
Odabir mikroupravljača za projektni zadatak

Prije izrade ovog projekta odlučeno je da će se koristiti ATmega32u4 zbog prethodno stečenih znanja o tom mikroračunalu na kolegiju mikroračunala.

Karakteristike mikroračunala ATmega32u4 su sljedeće:

- 32 8-bitna registra opće namjene
- Do 16 MIPS (16 milijuna instrukcija u jednoj sekundi) na 16 MHz
- 32 kilobajta Flash memorije
- 1 kilobajt EEPROM memorije
- 2.5 kilobajta SRAM memorije
- Ciklus pisanja/brisanja: 10 000 Flash / 100 000 EEPROM
- JTAG sučelje
- 26 digitalnih ulaza i izlaza
- 12 analognih ulaza
- 12 prekidnih pinova
- Jedan 8-bitni tajmer/brojač
- Dva 16-bitna tajmera/brojača
- Jedan 10-bitni tajmer/brojač
- USB
- USART
- SPI
- IIC
- 44 pinsko QFN podnožje
- Radni napon: 2.7 - 5.5 V

Prilikom izrade projekta korišteno je 20 pinova od ukupno 26 dostupnih. Cijena za ATmega32u4 na bazi 1 do 24 uređaja je 5.40\$, a za 5000 ili više uređaja je 3.95\$.



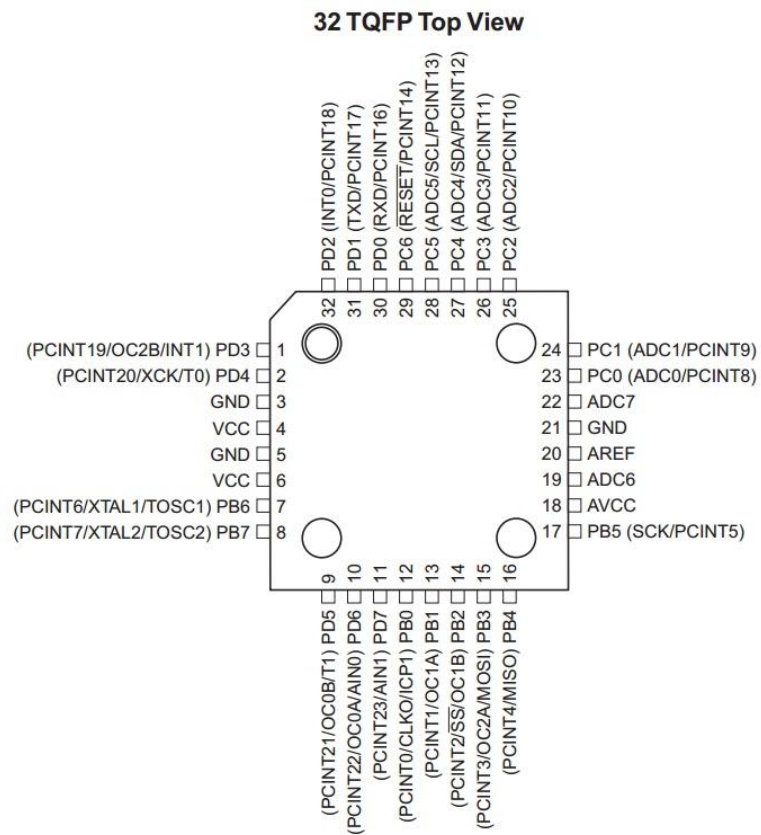
Slika 2.3. Raspored pinova na mikroračunalu ATmega32u4

Mikroračunalo koji bi najbolje zadovoljavao uvjete za ovaj projekt je ATmega328P. Cijena ATmega328P na bazi 1 do 24 uređaja je 2.80\$, a za 5000 ili više uređaja je 2.02\$.

Karakteristike mikroračunala ATmega328P su sljedeće:

- 32 8-bitna registra opće namjene
- Do 16 MIPS (16 milijuna instrukcija u jednoj sekundi) na 16 MHz
- 32 kilobajta Flash memorije
- 1 kilobajt EEPROM memorije
- 2 kilobajta SRAM memorije
- Ciklus pisanja/brisanja: 10 000 Flash / 100 000 EEPROM
- 23 digitalnih ulaza i izlaza
- 8 analognih ulaza
- 23 prekidnih pinova
- Dva 8-bitna tajmera/brojača
- Jedan 16-bitni tajmer/brojač
- USART
- SPI
- IIC
- 32 pinsko TQFP podnožje
- Radni napon: 2.7 - 5.5 V

Na slici 2.2. prikazan je raspored pinova na mikroračunalu ATmega328P



Slika 2.4. Raspored pinova na mikroračunalu ATmega328P

3. Opis hardvera

3.1 Tehničke specifikacije mikroupravljača ATmega32u4

Za izradu projekta korišteno je mikroračunalo ATmega32u4. ATmega32u4 je 8-bitno mikroračunalo tvrtke ATMEL. Ima 135 instrukcija i 32 8-bitna registra opće namjene. Memorijski gledano ima 32 kilobajta programske (eng. Flash) memorije i 1 kilobajt EEPROM memorije. Ciklus pisanja/brisanja određuje životni vijek mikroračunala, te ATmega32u4 ostvaruje 10 000 pisanja/brisanja na Flash memoriju i 100 000 na EEPROM.

ATmega32u4 se sastoji od 44 pinova. Na mikroračunalu se nalazi 26 digitalnih ulaza i izlaza koji su smješteni na 5 portova:

- PORTB (PB0, PB1, PB2, PB3, PB4, PB5, PB6, PB7)
- PORTC (PC6, PC7)
- PORTD (PD0, PD1, PD2, PD3, PD4, PD5, PD6, PD7)
- PORTE (PE2, PE6)
- PORTF (PF0, PF1, PF4, PF5, PF6, PF7)

Na tablici 3.1. je lista korištenih pinova mikroračunala za ovaj projekt.

Tablica 3.1.: Lista korištenih pinova mikroračunala

PORT	Opis
PORTB 0-3	USBasp
PORTB 4	HC-SR04 (Lijevi) - ECHO
PORTB 5	HC-SR04 (Centralni) - ECHO
PORTB 6	HC-SR04 (Desni) - ECHO
PORTB 7	Zujalica
PORTC 6	LCD - D5
PORTC 7	LCD - D4
PORTD 2	HC-SR04 (Lijevi) - TRIG
PORTD 3	HC-SR04 (Centralni) - TRIG
PORTD 4	HC-SR04 (Desni) - TRIG

PORTD 6	LCD - D7
PORTD 7	LCD - D6
PORTF 0	LCD - RS
PORTF 1	LCD - E
PORTF 5	Zelena LED dioda
PORTF 6	Žuta LED dioda
PORTF 7	Crvena LED dioda

3.2 USBasp

Za programiranje mikroračunala koristi se programator USBasp koji je prikazan na slici 3.1. Programator je spojen na konektor za programiranje. Programiranje se izvodi putem SPI protokola čije sučelje se sastoji od pinova SS (PB0), SCLK (PB1), MOSI (PB2) i MISO (PB3).



Slika 3.1. USBasp programator

3.3 Ultrazvučni senzor HC-SR04

HC-SR04 je senzor koji se u projektu koristi za mjerenje udaljenosti od prepreke. U programskom kodu se uzima najbliža udaljenost od prepreke i na temelju toga se uključuje LED dioda. Ako je udaljenost od prepreke manja od 200 cm, a veća od 60 cm, to je 1. zona i uključuje se zelena LED dioda. Ako je udaljenost manja od 60 cm i veća od 20 cm, to je

2. zona i uključuje se žuta LED dioda. Ako je udaljenost manja od 20 cm i veća od 0 cm, to je 3. zona i uključuje se crvena LED dioda.

TRIG pinovi su spojeni na PD2, PD3 i PD4, a ECHO pinovi su spojeni na PB4, PB5, PB6. Ultrazvučni senzor u ovom projektu radi tako da TRIG pin generira impuls koji traje 10 μ s nakon čega se aktivira ECHO pin, te pomoću tajmera 1 mjeri koliko je dugo bio taj ECHO pin u visokom stanju. To se sve događa u PCINT0 (eng. pin change interrupt 0). Nakon toga se iz izmjerenog vremena može izračunati udaljenost za taj određeni senzor. Prvo trebamo izračunati t_{echo} , vrijeme trajanja ECHO impulsa koje predstavlja ukupno vrijeme od senzora do prepreke i nazad.

Računa se relacijom:

$$t_{\text{echo}} = \text{broj_impulsa} * 8.0 / F_{\text{CPU}}$$

F_{CPU} je definirani radni takt mikroračunala, u projektu se koristi 16MHz.

Nakon dobivenog t_{echo} računamo stvarnu udaljenost senzora od prepreke relacijom:

$$d = t_{\text{echo}} / 2.0 * 343.0 * 100.0$$



Slika 3.2. HC-SR04

3.4 LCD ekran (GDM1602E)

LCD ekran se koristi u projektu za prikaz svih udaljenosti koje senzori očitavaju. Ovaj LCD ekran ima 16 stupaca i 2 redka.

Pin no.	Symbol	External connection	Function
1	V _{SS}	Power supply	Signal ground for LCM (GND)
2	V _{DD}		Power supply for logic for LCM
3	V ₀		Contrast adjust
4	RS	MPU	Register select signal
5	R/W	MPU	Read/write select signal
6	E	MPU	Operation (data read/write) enable signal
7~10	DB0~DB3	MPU	Four low order bi-directional three-state data bus lines. Used for data transfer between the MPU and the LCM. These four are not used during 4-bit operation.
11~14	DB4~DB7	MPU	Four high order bi-directional three-state data bus lines. Used for data transfer between the MPU
15	A(LED+)	LED BKL power supply	Power supply for BKL (Anode)
16	K(LED-)		Power supply for BKL (GND)

Slika 3.3. Prikaz pinova LCD ekrana GDM1602E

LCD ekran prikazuje izmjerenu udaljenost za lijevi, centralni i desni senzor u stvarnom vremenu. Slika 3.4. prikazuje kako to izgleda na ekranu.



Slika 3.4. Prikaz udaljenosti senzora od prepreke na LCD ekranu



Slika 3.5. LCD ekran GDM1602E

3.5 Zujalica (eng. buzzer)

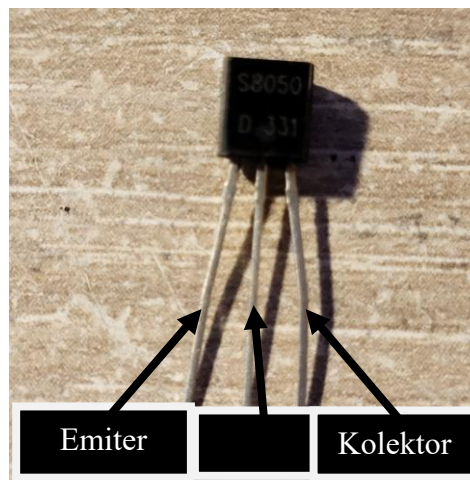
U projektu se koristi aktivna zujalica zbog jednostavnosti. Aktivna zujalica ima ugrađeni oscilator pa za ostvarenje zvuka se treba dovesti u određeno stanje. U ovom projektu zujalica se aktivira kada je u niskom stanju jer se koristi NPN tranzistor koji je konfiguriran na takav način. Zujalica mijenja ritam ovisno o trenutnoj zoni.



Slika 3.6. Prikaz aktivne zujalice

3.6 NPN tranzistor S8050

Za pravilan rad aktivne zujalice koristimo NPN tranzistor S8050. Na emiter je spojen GND, a na kolektor je spojeno 5V. Na bazu je spojen otpornik od 10 k Ω i onda nakon toga žica koja se spaja na PB7.



Slika 3.7. Prikaz NPN tranzistora S8050

3.7 LED dioda

LED diode se koriste u projektu za signalizaciju zona, 1. zona je zelena dioda, 2. zona je žuta dioda i 3. zona je crvena dioda.

Prije spajanja dioda u pinove mikroračunala stavljani su otpornici od 250Ω .



Slika 3.8. Prikaz LED diode

3.8 Troškovnik

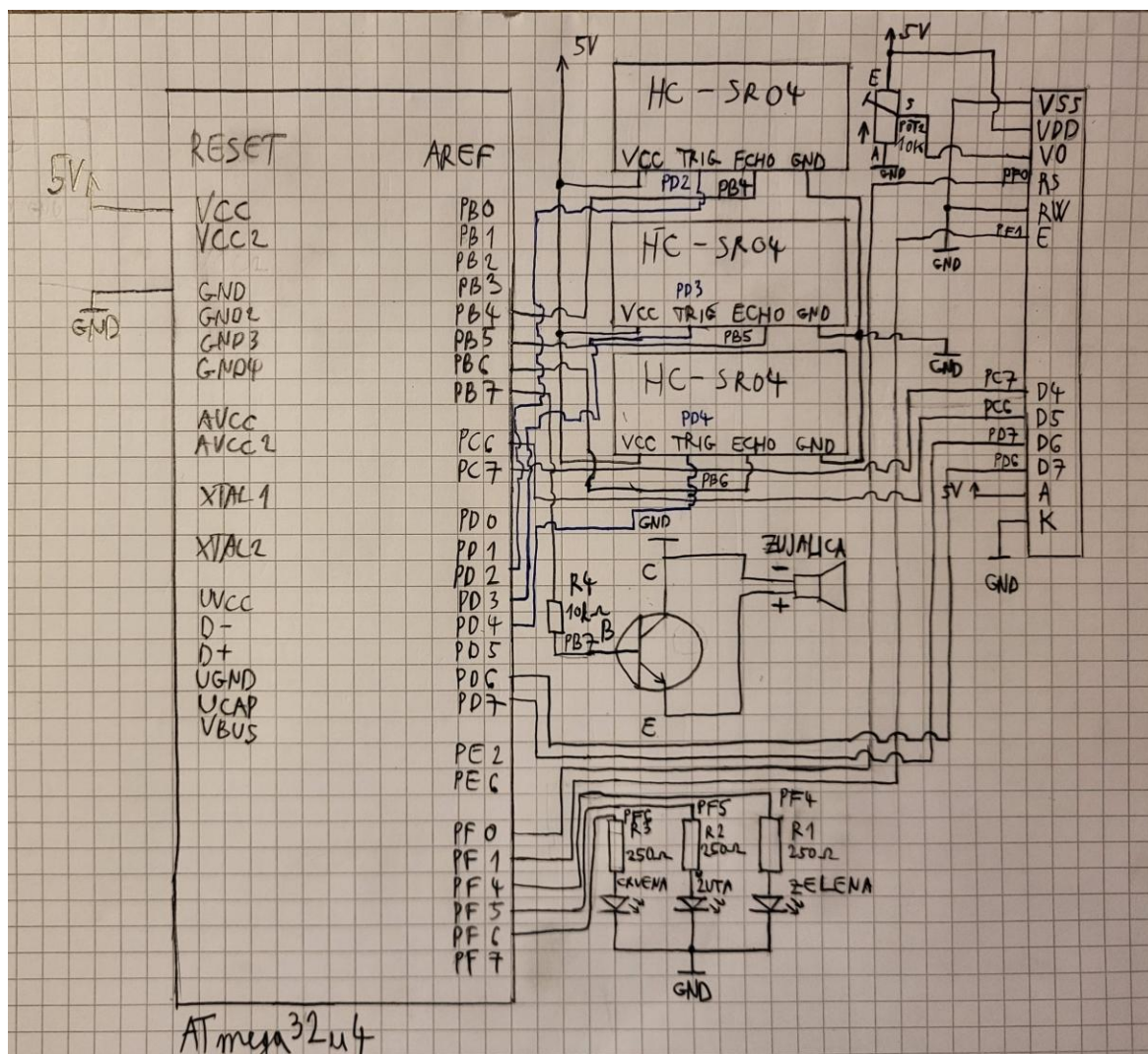
U tablici 3.2. su prikazane cijene komponenti korištenih u projektu.

Komponenta	Cijena (EUR)
ATmega32u4	4.60
USBasp	2.70
LCD ekran GDM1602E	1.70
HC-SR04 (3 kom.)	11.85
Zujalica	0.75
Paket 10mm LED dioda (13 kom.)	2.50
Paket otpornika (300 kom.)	2.92
Paket NPN tranzistori S8050 (20 kom.)	1.82
Paket žica (140 kom.)	2.39
Breadboard (2 kom.)	6.10
Ukupno	37.33

Tablica 3.2. Troškovnik

4. Električna shema uređaja

Shema se sastoji od 3 ultrazvučna senzora HC-SR04, LCD ekrana GDM1602E, aktivne zujalice, NPN tranzistora S8050, jednog 10 k Ω otpornika, 3 LED dioda i 3 250 Ω otpornika. Električnu shemu sam izradio ručno, plave žice su TRIG portovi ultrazvučnih senzora.



Slika 4.1. Električna shema parking senzora

5. Programski kod

Programski kod napisan je koristeći Microchip studio. Program eXtreme Burner AVR korišten je za slanje strojnog koda na mikroračunalo preko USBasp programatora. Fuse bitovi su izračunati preko web stranice Engbedded Atmel AVR Fuse Calculator. Ovaj projekt koristi postavke prikazane u slici 5.1., te to odgovara sljedećim fuse bitovima:

- Low fuse: 0xFF
- High fuse: 0xD9
- Extended fuse: 0xFB

Device selection

Select the AVR device type you want to configure. When changing this setting, default fuse settings will automatically be applied. Presets (hexadecimal representation of the fuse settings) can be reviewed and even be set in the last form at the bottom of this page.

AVR part name: (141 parts currently listed)

Feature configuration

This allows easy configuration of your AVR device. All changes will be applied instantly.

Features

Ext. Crystal Osc.; Frequency 8.0- MHz; Start-up time: 16K CK + 65 ms; [CKSEL=1111 SUT=11]

☐ Clock output on PORTC7; [CKOUT=0]

☐ Divide clock by 8 internally; [CKDIV8=0]

☐ Boot Reset vector Enabled (default address=\$0000); [BOOTRST=0]

Boot Flash size=2048 words start address=\$3800; [BOOTSZ=00] ; default value

☐ Preserve EEPROM memory through the Chip Erase cycle; [EESAVE=0]

☐ Watchdog timer always on; [WDTON=0]

☒ Serial program downloading (SPI) enabled; [SPIEN=0]

☐ JTAG Interface Enabled; [JTAGEN=0]

☐ On-Chip Debug Enabled; [OCDEN=0]

Brown-out detection level at VCC=2.6 V; [BODLEVEL=011]

☐ Hardware Boot Enable; [HWBE=0]

Apply feature settings

Slika 5.1. Postavke fuse bitova

Radni takt koji se koristi za ovaj projekt je 16MHz.

Programski kod 5.1

```

/*
Naslov: Parking senzori
Autor: Matko Manzin
*/

#include <avr/io.h>
#include "AVR VUB/avrvub.h"
#include <util/delay.h>
#include "LCD/lcd.h"
#include "Timer/timer.h"
#include "Interrupt/interrupt.h"
#include "ADC/adc.h"

volatile uint32_t broj_impulsa1 = 0; // broj impulsa tajmera 1 (trajanje ECHO impulsa) za senzor 1, za izracun udaljenosti
volatile uint32_t broj_impulsa2 = 0; // broj impulsa tajmera 1 (trajanje ECHO impulsa) za senzor 2, za izracun udaljenosti
volatile uint32_t broj_impulsa3 = 0; // broj impulsa tajmera 1 (trajanje ECHO impulsa) za senzor 3, za izracun udaljenosti
volatile bool hcsr04_measured1 = false; // pomocna varijabla, da znamo ako je mjerenje senzora 1 gotovo da bismo izacunali udaljenost
volatile bool hcsr04_measured2 = false; // pomocna varijabla, da znamo ako je mjerenje senzora 2 gotovo da bismo izacunali udaljenost
volatile bool hcsr04_measured3 = false; // pomocna varijabla, da znamo ako je mjerenje senzora 3 gotovo da bismo izacunali udaljenost
volatile uint8_t echo1_high = 0; // stanje ECHO signala za senzor 1, da znamo kada je signal u visokom stanju (1), a kada u niskom (0)
volatile uint8_t echo2_high = 0; // stanje ECHO signala za senzor 2, da znamo kada je signal u visokom stanju (1), a kada u niskom (0)
volatile uint8_t echo3_high = 0; // stanje ECHO signala za senzor 3, da znamo kada je signal u visokom stanju (1), a kada u niskom (0)

// predprocesorske direktive
#define LED_GREEN PF5 //PF5 je isto kao i LED_GREEN
#define LED_YELLOW PF6 //PF6 je isto kao i LED_YELLOW
#define LED_RED PF7 //PF7 je isto kao i LED_RED
#define BUZZER PB7 //PB7 je isto kao i BUZZER

// varijable za rad zujalice (BUZZER)
static uint8_t buzz_on = 0; // stanje zujalice, 0 - iskljuceno, 1 - ukljuceno
static uint16_t buzz_phase_ms = 0; // proteklo vrijeme rada zujalice u trenutnoj fazi, ms
static uint16_t buzz_on_ms = 0; // vrijeme koliko zujalica treba raditi u ukljucenoj fazi, ms
static uint16_t buzz_off_ms = 0; // vrijeme koliko zujalica treba raditi u iskljucenoj fazi, ms
static uint8_t last_zone = 0xFF; // koja je zona bila zadnja aktivna, inicijalizirano na 0xFF

ISR(PCINT0_vect) // prekidna rutina PCINT0, reagira na promjenu stanja pinova PB4, PB5, PB6
{
    uint8_t pins = PINB; // sprema trenutno stanje svih pinova PORTB

    // senzor 1 (lijevi)
    if (pins & (1 << PB4)) { // ako je PB4 u visokom stanju, samo ako je bio poslan TRIG na PD2
        if (echo1_high == 0) { // ako ECHO nije bio u visokom stanju (nije mjerio udaljenost)
            echo1_high = 1; // postaviti ECHO u visoko stanje
            TCNT1 = 0; // resetiranje tajmera 1, početak mjerenja
        }
    }
}

```

```

    }
    } else { // ako PB4 nije u visokom stanju
        if (echo1_high == 1) { // ako je ECHO bio u visokom stanju (mjerio
            udaljenost)
                echo1_high = 0; // postaviti ECHO u nisko stanje (završiti
            mjerenje udaljenosti)
                broj_impulsa1 = TCNT1; // spremanje broja impulsa tajmera 1,
            služi za računanje t_echo varijable
                hcsr04_measured1 = true; // pomocna varijabla, mjerenje senzora
            1 je gotovo i može se računati udaljenost
        }
    }
    // senzor 2 (centralni), sve radi na isti način kao i senzor 1
    if (pins & (1 << PB5)) {
        if (echo2_high == 0) {
            echo2_high = 1;
            TCNT1 = 0;
        }
        } else {
            if (echo2_high == 1) {
                echo2_high = 0;
                broj_impulsa2 = TCNT1;
                hcsr04_measured2 = true;
            }
        }
    // senzor 3 (desni), sve radi na isti način kao i senzor 1
    if (pins & (1 << PB6)) {
        if (echo3_high == 0) {
            echo3_high = 1;
            TCNT1 = 0;
        }
        } else {
            if (echo3_high == 1) {
                echo3_high = 0;
                broj_impulsa3 = TCNT1;
                hcsr04_measured3 = true;
            }
        }
    }
}

void inicijalizacija()
{
    lcd_init(); // inicijalizacija lcd ekrana, makronaredbe

    // tajmer 1 je postavljen u normalan način rada, WGM bitovi = 0 0 0
    TCCR1A &= ~(1 << WGM00) | (1 << WGM01); // WGM00 = 0, WGM01 = 0, TCCR1A
    registar
    TCCR1B &= ~(1 << WGM02); // WGM02 = 0, TCCR1B registar

    // prescaler_8, CS01 mora biti 1, očitano iz tablice u knjizi
    TCCR1B &= ~(1 << CS02) | (1 << CS01) | (1 << CS00); // postavljanje svih CS
    bitova u 0, TCCR1B registar
    TCCR1B |= (1 << CS01); // postavljanje CS01 bita u 1

    // TRIG pinovi postavljeni kao izlazi
    DDRD |= (1 << PD2) | (1 << PD3) | (1 << PD4);

    // ECHO pinovi postavljeni kao ulazi
    DDRB &= ~(1 << PB4) | (1 << PB5) | (1 << PB6));
    PORTB &= ~(1 << PB4) | (1 << PB5) | (1 << PB6)); // isključeni su pull-up
    otpornici

```

```

    // omoguceni su pin-change prekidi za PB4, PB5, PB6
    PCMSK0 |= (1 << PCINT4) | (1 << PCINT5) | (1 << PCINT6);
    PCICR |= (1 << PCIE0); // omogucen je PCINT0 prekid

    // LED pinovi postavljeni kao izlazi, PF5, PF6, PF7
    DDRF |= (1 << LED_GREEN) | (1 << LED_YELLOW) | (1 << LED_RED);
    PORTF &= ~(1 << LED_GREEN) | (1 << LED_YELLOW) | (1 << LED_RED)); // pocetno
stanje svih LED dioda je iskljuceno

    // BUZZER pin, PB7 postavljen kao izlaz
    DDRB |= (1 << BUZZER);
    PORTB |= (1 << BUZZER); // PB7, zujalicu je potrebno staviti u nisko stanje
da se ukljuci, tako da je sada u visokom (iskljucenom) stanju

    sei(); // globalno omoguceni prekidi
}

// metoda za generiranje TRIG impulsa za odabrani senzor prema 'id' parametru
void hcsr04_trigg(uint8_t id) // potrebno proslijediti broj kao parametar kod
pozivanja metode
{
    switch (id) // odabire switch-case ovisno o proslijedenom broju u parametru
    {
        case 0: // ako je proslijeden broj 0
            PORTD |= (1 << PD2); // TRIG pin, PD2, senzora 1 ide u visoko stanje
            (pocetak TRIG impulsa)
            _delay_us(10); // u visokom stanju je 10 mikrosekundi
            PORTD &= ~(1 << PD2); // TRIG pin, PD2, senzora 1 ide nazad u nisko
stanje (kraj TRIG impulsa)
            break;

        case 1: // ako je proslijeden broj 1
            PORTD |= (1 << PD3); // TRIG pin, PD3, senzora 2 ide u visoko stanje
            (pocetak TRIG impulsa)
            _delay_us(10); // u visokom stanju je 10 mikrosekundi
            PORTD &= ~(1 << PD3); // TRIG pin, PD3, senzora 2 ide nazad u nisko
stanje (kraj TRIG impulsa)
            break;

        case 2: // ako je proslijeden broj 2
            PORTD |= (1 << PD4); // TRIG pin, PD4, senzora 3 ide u visoko stanje
            (pocetak TRIG impulsa)
            _delay_us(10); // u visokom stanju je 10 mikrosekundi
            PORTD &= ~(1 << PD4); // TRIG pin, PD4, senzora 3 ide nazad u nisko
stanje (kraj TRIG impulsa)
            break;
    }
}

// metoda za odabir ritma zujalice ovisno o proslijedenom parametru 'zone'
static void buzzer_set_pattern(uint8_t zone) { // potrebno proslijediti broj kao
parametar kod pozivanja metode

    // zona: 0 = iskljuceno, 1 = zelena zona; sporo, 2 = zuta zona; ubrzano, 3 =
crvena zona; jako brzo
    if (zone == 1) { // ako je proslijeden broj 1, zelena zona, sporo
        buzz_on_ms = 500; // zujalica mora biti upaljena 500 milisekundi
        buzz_off_ms = 100; // zujalica mora biti iskljucena 100 milisekundi
    } else if (zone == 2) { // ako je proslijeden broj 2, zuta zona,
ubrzano
        buzz_on_ms = 300; // zujalica mora biti upaljena 300 milisekundi
        buzz_off_ms = 90; // zujalica mora biti iskljucena 90 milisekundi
    }
}

```

```

        } else if (zone == 3) { // ako je proslijeden broj 3, crvena zona,
jako brzo
        buzz_on_ms = 120; // zujalica mora biti upaljena 120 milisekundi
        buzz_off_ms = 80; // zujalica mora biti isključena 80 milisekundi
        } else { // ako je proslijeden broj 0, isključena je zujalica
        buzz_on_ms = 0; // zujalica mora biti upaljena 0 milisekundi
        buzz_off_ms = 0; // zujalica mora biti isključena 0 milisekundi
        }

        if (buzz_on_ms > 0) { // ako je vrijeme kada zujalica mora biti uključena
veća od 0
        buzz_on = 1; // zujalica mora biti uključena
        } else {
        buzz_on = 0; // ako vrijeme nije veće od 0, onda mora biti isključena
        }

        buzz_phase_ms = 0; // brojac trajanja trenutne faze

        if (buzz_on == 0) { // ako je stanje zujalice isključeno
        PORTB |= (1 << BUZZER); // postavi PB7 u visoko stanje (isključiti
buzzer)
        }
    }

    // metoda za upravljanje stanjem zujalice
    static void buzzer_update(uint16_t dt_ms) { // potrebno je proslijediti broj kao
parametar
        if (buzz_on_ms == 0 && buzz_off_ms == 0) { // ako je vrijeme kada zujalica
treba biti uključena i isključena jednaka 0
        PORTB |= (1 << BUZZER); // postaviti PB7 u visoko stanje
        return;
        }
        if (buzz_on == 1) { // ako je stanje zujalice 1
        PORTB &= ~(1 << BUZZER); // postaviti PB7 u nisko stanje (uključiti
zujalicu)
        } else {
        PORTB |= (1 << BUZZER); // ako je stanje zujalice 0, PB7 postaviti u
visoko stanje
        }

        buzz_phase_ms += dt_ms; // vrijeme provedeno u fazi poprima vrijednost
proslijeđenog parametra dt_ms

        if (buzz_on == 1) { // ako je stanje zujalice 1, uključena
        if (buzz_phase_ms >= buzz_on_ms) { // i ako je vrijeme provedeno u
trenutnoj fazi veće ili jednako vremenu kada zujalica mora biti uključena (ritam)
        buzz_phase_ms = 0; buzz_on = 0; // postavi 'buzz_phase_ms'
nazad u 0 i napravi toggle 'buzz_on' varijable (isključiti zujalicu)
        }
        } else { // ako je stanje zujalice 0, isključena
        if (buzz_phase_ms >= buzz_off_ms) { // i ako je vrijeme provedeno u
trenutnoj fazi veće ili jednako vremenu kada zujalica mora biti isključena (ritam)
        buzz_phase_ms = 0; buzz_on = 1; // postavi 'buzz_phase_ms'
nazad u 0 i napravi toggle 'buzz_on' varijable (uključiti zujalicu)
        }
        }
    }
}

int main(void)
{
    inicijalizacija();
}

```

```

float d1 = 0, d2 = 0, d3 = 0; // varijable za udaljenost
float t_echo; // vrijeme trajanja ECHO impulsa koje predstavlja ukupno
vrijeme od senzora do prepreke i nazad
uint16_t time_delay = 0; // pomocna varijabla koja sluzi za sekvencijalno
aktiviranje senzora svakih 50 ms
uint8_t koji = 0; // varijabla za odabir aktivnog senzora, sluzi za
sekvencijalno pobudivanje

while (1) {
    // sekvencijalno aktiviranje senzora
    if (time_delay == 0) { // ako je prva iteracija ili je proslo 50 ms,
aktiviraj TRIG port određenog ultrazvucnog senzora
        hcsr04_trigg(koji); // metoda za aktiviranje TRIG senzora 10
mikrosekundi
        koji = (koji + 1) % 3; // odabir 'id' sljedeceg senzora (0 -> 1
-> 2 -> 0 -> ...)
    }
    if (hcsr04_measured1 == true) { // ako je senzor 1 ECHO pin otisao iz
visokog u nisko stanje, zavrсило je mjerenje za taj senzor
        t_echo = broj_impulsa1 * 8.0 / F_CPU; // FORMULA trajanja ECHO
signala: broj_impulsa * 8.0 / F_CPU
        d1 = t_echo / 2.0 * 343.0 * 100.0; // FORMULA udaljenosti do
prepreke: t_echo / 2.0 * 343.0 * 100, 343.0 je brzina zvuka na 20 stupnjeva
        hcsr04_measured1 = false; // nakon izracunate udaljenosti,
resetiranje varijable da bi se omogucio daljni rad ovog senzora
    }
    if (hcsr04_measured2 == true) { // sve radi na isti nacin kao i senzor
1
        t_echo = broj_impulsa2 * 8.0 / F_CPU;
        d2 = t_echo / 2.0 * 343.0 * 100.0;
        hcsr04_measured2 = false;
    }
    if (hcsr04_measured3 == true) { // sve radi na isti nacin kao i senzor
1
        t_echo = broj_impulsa3 * 8.0 / F_CPU;
        d3 = t_echo / 2.0 * 343.0 * 100.0;
        hcsr04_measured3 = false;
    }

    static uint16_t lcd_brojac = 0; // brojac za osvježavanje LCD ekrana

    lcd_brojac += 10; // povecava brojac za 10 jer petlja traje 10 ms

    if (lcd_brojac >= 200) { // kada brojac dode do 200, nakon 200 ms
        lcd_brojac = 0; // resetiranje brojaca

        lcd_clrscr(); // obrisi sve na ekranu
        lcd_home(); // postaviti pokazivac u prvi redak, prvi stupac
(0, 0)

        lcd_gotoxy(0, 0); // postaviti pokazivac na koordinate x = 0, y
= 0 (1. red, 1. stupac)
        lcd_print("L:%0.1f D:%0.1f", d1, d3); // ispisi na ekranu
informaciju o udaljenosti

        lcd_gotoxy(1, 0); // postaviti pokazivac na koordinate x = 1, y
= 0 (2. red, 1. stupac)
        lcd_print("C:%0.1f", d2); // ispisi na ekranu informaciju o
udaljenosti
    }

    // konfiguracija zona

```

```

        float min_dist = d1; // inicijalno najmanja udaljenost je lijevi
senzor
        if (d2 < min_dist) {
            min_dist = d2; // ako centralni senzor ima manju udaljenost od
prepreke od lijevog senzora onda se uzima njegova udaljenost
        }
        if (d3 < min_dist) {
            min_dist = d3; // i ako desni senzor ima manju udaljenost od
centralnog senzora onda se uzima njegova udaljenost
        }

        uint8_t zone = 0; // inicijalno zona je 0

        if (min_dist <= 200.0f && min_dist >= 60.0f) { // ako je najmanja
udaljenost manja od 200 cm i veca od 60 cm
            zone = 1; // zona je 1, zelena zona
        }
        else if (min_dist < 60.0f && min_dist >= 20.0f) { // ako je najmanja
udaljenost manja od 60 cm i veca od 20 cm
            zone = 2; // zona je 2, zuta zona
        }
        else if (min_dist < 20.0f && min_dist >= 0.0f) { // ako je najmanja
udaljenost manja od 20 cm i veca od 0 cm
            zone = 3; // zona je 3, crvena zona
        }

        PORTF &= ~( (1 << LED_GREEN) | (1 << LED_YELLOW) | (1 << LED_RED)); //
postavi sve LED diode u 0

        if (zone == 1) {
            PORTF |= (1 << LED_GREEN); // ako je zona 1 onda se upali
zelena dioda
        } else if (zone == 2) {
            PORTF |= (1 << LED_YELLOW); // ako je zona 2 onda se upali zuta
dioda
        } else if (zone == 3) {
            PORTF |= (1 << LED_RED); // ako je zona 3 onda se upali crvena
dioda
        }

        // odredjivanje ritma zujalice ovisno o zoni
        if (zone != last_zone) { // ako je trenutna zona razlicita od prosle
zone
            buzzer_set_pattern(zone); // promjeniti ritam zujalice u tu
novu zonu
            last_zone = zone; // trenutna zona je sada prosla zona
        }

        buzzer_update(10); // dodaje 10 ms u 'buzz_phase_ms' varijablu i
svakom iteracijom petlje se 'buzz_phase_ms' povecava i kada dosegne zadano trajanje
faze prelazi u sljedeću fazu

        _delay_ms(10); // postavljeno kasnjenje petlje na 10 ms

        time_delay++; // povecaj pomocnu varijablu za sekvencijsko aktiviranje
senzora
        if (time_delay > 4) { // ako je ovo 5. iteracija petlje
            time_delay = 0; // resetiraj pomocnu varijablu 'time_delay'
koja služi za sekvencijsko aktiviranje senzora
        }
    }
}

```

6. Zaključak

Izradom ovog projekta proširena su znanja o ultrazvučnim senzorima, vanjskim prekidima i tajmerima. Projekt je ostvario svoju svrhu, a to su parking senzori koji nude vizualnu povratnu informaciju preko LED dioda u zonama (zelena, žuta, crvena) zajedno s akustičnom povratnom informacijom preko aktivne zujalice koja mijenja ritam zujanja ovisno o udaljenosti. U odnosu na parking senzore automobila, prednost ovog projekta je potpuna kontrola. Moguće je mijenjati ritam zujalice i definirati nove pragove zona. Također ovo je jeftinije od automobilskih parking senzora, samo nije toliko precizno i robusno na vanjske uvjete (kiša, vlaga, vrućina, hladnoća).

7. Literatura

Vrhovski, Z. (2020). *Mikroupravljači - Programiranje mikroupravljača porodice AVR*.
Bjelovar: Veleučilište u Bjelovaru.